
Cloud & DevOps System Design Document

Document System Design Document (SDD)
Position Cloud & DevOps Intern – CI/CD & Infrastructure
Company KAMKA IT
Candidate chouchane omar
Version 1.0
Status Draft
Date June 5, 2026

Next.js | Node.js | PostgreSQL | Docker | GitHub Actions | GHCR
AWS EC2 | Terraform | Caddy | Prometheus + Grafana | Trivy + SonarQube

Internal document. Stack: containerised three-tier application deployed via automated CI/CD pipeline to AWS EC2, with HTTPS reverse proxy, OIDC auth, and live observability.

Contents

1	System Overview	1
1.1	Purpose	1
1.2	Platform at a Glance	1
2	Objectives	2
3	Architecture Pattern	3
3.1	Topology	3
3.2	System Architecture Diagram	3
4	Infrastructure Design	4
4.1	AWS Resources	4
4.2	AWS Infrastructure Diagram	4
4.3	Terraform Structure	4
5	CI/CD Design	5
5.1	Pipeline Overview	5
5.2	CI/CD Pipeline Flow	5
5.3	Pipeline Stages	5
5.4	OIDC Authentication	5
6	Security Design	6
6.1	Threat Model Summary	6
6.2	Secrets Handling	6
6.3	IAM Least Privilege	6
7	Deployment Strategy	7
7.1	Rolling Update via Docker Compose	7
7.2	Rolling Update Flow Diagram	7
7.3	Rollback Procedure	7
8	Observability	8
8.1	Stack	8
8.2	Key Alerts	8
8.3	Health Endpoints	8
9	Operational Procedures	9
9.1	Deploy	9
9.2	Rollback	9
9.3	Database Backup	9
9.4	Bootstrap	9
10	Constraints & Trade-offs	10
11	Future Improvements	11

1. System Overview

1.1 Purpose

This document covers the architecture, infrastructure, CI/CD pipeline, security model, deployment strategy, and observability setup for a production-grade cloud-native platform deployed on AWS EC2. It is scoped for a small team and designed to keep operational overhead low.

1.2 Platform at a Glance

Layer	Technology	Role
Frontend	Next.js (React)	SSR web application
Backend	Node.js / Express	REST API
Database	PostgreSQL 15	Persistent data store
Containerisation	Docker + Docker Compose	Runtime isolation
Reverse Proxy	Caddy	TLS termination, routing
CI/CD	GitHub Actions	Automation pipeline
Registry	GHCR	Container image store
Cloud	AWS EC2 (t3.micro, Elastic IP)	Compute host
IaC	Terraform (dev / prod modules)	Infrastructure provisioning
Auth	OIDC (GitHub → AWS IAM)	Keyless deployment auth
Scanning	Trivy + SonarQube	Security & quality gates
Observability	Prometheus + Grafana	Metrics and dashboards
Alerting	Alertmanager + Slack	Incident notification
Security reports	Trivy SMTP	Email vulnerability reports

2. Objectives

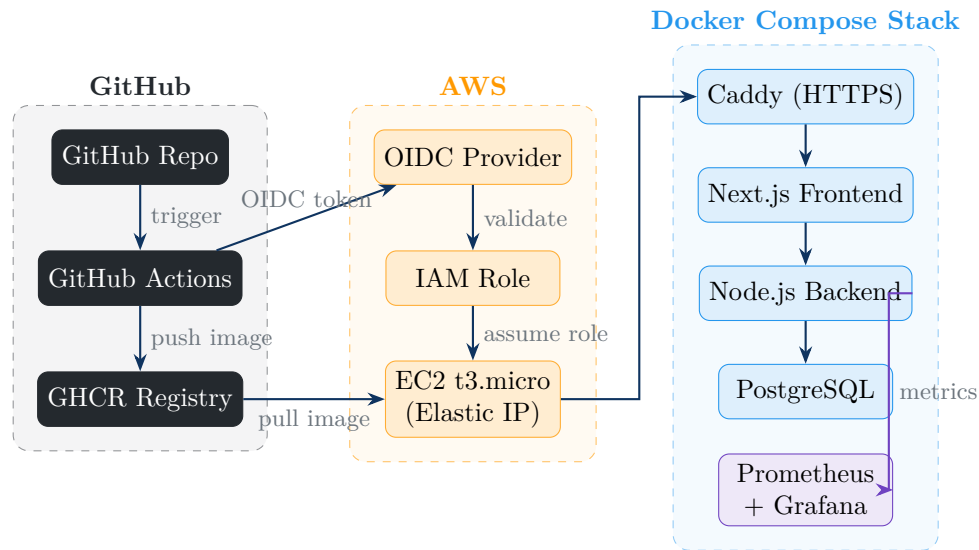
- Deploy a containerised three-tier application on a single EC2 instance using Docker Compose.
- Eliminate long-lived AWS credentials through GitHub OIDC federation.
- Enforce automated security gates (SAST, SCA, image scanning) on every commit.
- Send Trivy vulnerability reports by email after every scan.
- Achieve near-zero-downtime deployments through rolling container updates.
- Provide full metrics visibility via Prometheus, Grafana, and Alertmanager.
- Manage all cloud resources as code (Terraform) with separate **dev** and **prod** workspaces.
- Keep operational complexity low enough for a team of fewer than five engineers.

3. Architecture Pattern

3.1 Topology

Single-host container orchestration (Docker Compose on EC2) behind a Caddy reverse proxy. The pattern is **monorepo + single-node deployment**. Kubernetes migration is the documented future path.

3.2 System Architecture Diagram

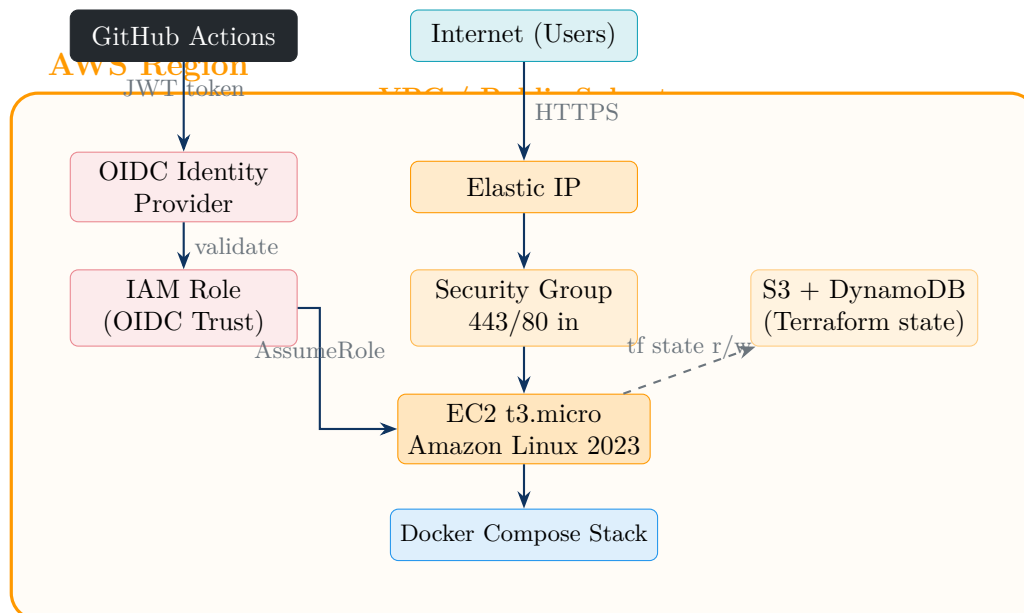


4. Infrastructure Design

4.1 AWS Resources

- **VPC** with a single public subnet (CIDR 10.0.0.0/24).
- **EC2 instance** (t3.micro, Amazon Linux 2023) with an attached Elastic IP for a stable public address.
- **Security Group** – inbound: 443/tcp (world), 80/tcp (redirect to HTTPS), 22/tcp (restricted CIDR only); outbound: unrestricted.
- **IAM Role** federated via GitHub OIDC. Policy grants only the minimum permissions needed: pull from GHCR, read SSM parameters.
- **S3 bucket** (versioned) for Terraform remote state with a DynamoDB table for state locking.

4.2 AWS Infrastructure Diagram



4.3 Terraform Structure

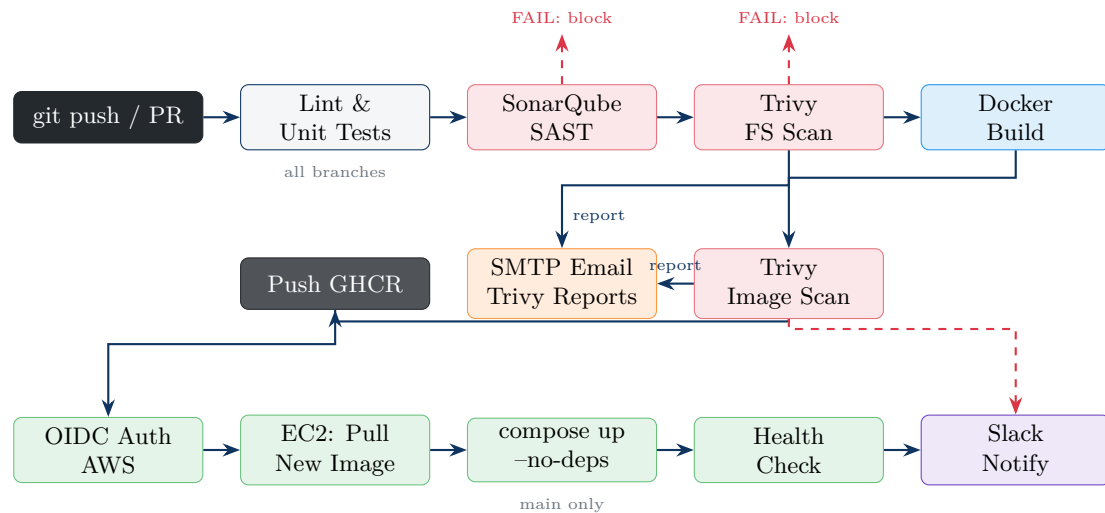
Terraform is organised into reusable modules (`vpc`, `ec2`, `security`, `iam`) called from separate `dev` and `prod` environment directories, each with its own `terraform.tfvars`. Remote state lives in S3; DynamoDB prevents concurrent state writes.

5. CI/CD Design

5.1 Pipeline Overview

Every push to any branch runs the quality and security jobs. Only a push to `main` triggers the deploy job. Any failing gate stops subsequent stages immediately.

5.2 CI/CD Pipeline Flow



5.3 Pipeline Stages

- **Lint & Unit Tests** – ESLint, Prettier, Jest. Runs on every push. Fast feedback; fails loudly.
- **SonarQube SAST** – Static code analysis for bugs, code smells, and security hotspots. Blocks merge on quality gate failure.
- **Trivy FS Scan** – Scans the repository filesystem for vulnerable dependencies and exposed secrets before any image is built.
- **Docker Build** – Multi-stage build; produces a minimal final image.
- **Trivy Image Scan** – Scans the built image for OS and library CVEs. Fails on HIGH/CRITICAL unfixed vulnerabilities.
- **SMTP Email – Trivy Reports** – Both the FS and image scan reports (JSON/table format) are sent via SMTP to the security inbox after every run, regardless of pass/fail outcome.
- **Push GHCR** – Tagged image pushed to GitHub Container Registry. Only runs if all scans pass.
- **OIDC Auth + Deploy** – GitHub exchanges an OIDC JWT for a short-lived AWS role credential; SSH into EC2; `docker compose up`.
- **Health Check** – Polls `/healthz` on each service; fails the job and triggers Slack alert if any service does not respond within 60 s.
- **Slack Notify** – Posts pipeline outcome (pass or fail) with a link to the run logs.

5.4 OIDC Authentication

GitHub Actions requests a short-lived JWT from the OIDC provider and exchanges it for temporary AWS credentials via `sts:AssumeRoleWithWebIdentity`. No static AWS access keys exist anywhere. The IAM trust policy restricts assumption to the specific repository and branch.

6. Security Design

6.1 Threat Model Summary

Threat	Control	Tool
Leaked AWS credentials	OIDC federation	GitHub OIDC + IAM
Vulnerable dependencies	Dependency scanning	Trivy FS
Vulnerable container base images	Image scanning	Trivy image
Code-level vulnerabilities	SAST	SonarQube
Exposed secrets in repo	Pre-build FS scan	Trivy secret scanner
Misconfigured IaC	Policy-as-code (optional)	Checkov
Exposed ports	Least-privilege security group	Terraform
Plaintext traffic	TLS everywhere	Caddy (auto-ACME)
Secrets in environment	Secret store	GitHub Secrets + SSM
Unreported vulnerabilities	Automated email reports	Trivy + SMTP

6.2 Secrets Handling

Secrets are never committed to the repository. Local development uses a `.env` file (gitignored), with a committed `.env.example` showing all required keys without values. The CI pipeline reads secrets from GitHub Actions encrypted secrets. On EC2, secrets are injected at deploy time from AWS SSM Parameter Store, which the IAM role has read-only access to.

6.3 IAM Least Privilege

The GitHub deploy role grants only three permissions: read from GHCR (via `ecr:GetAuthorizationToken`), read SSM parameters scoped to `/myapp/*`, and no write permissions of any kind to AWS resources. All other actions are explicitly denied by default.

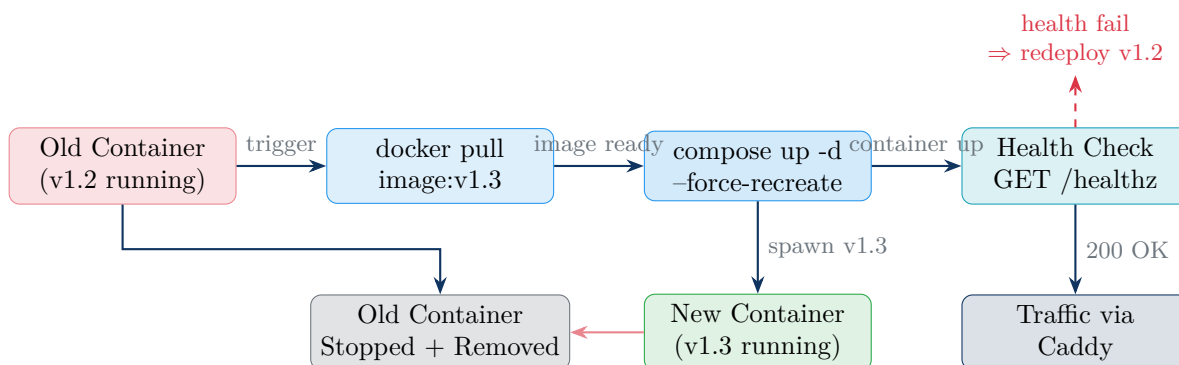
7. Deployment Strategy

7.1 Rolling Update via Docker Compose

No external load balancer is used. Caddy buffers upstream connections during the brief container restart window (approximately 5–10 seconds per service). The sequence for a rolling update is:

1. Pull the new image tag on EC2.
2. Run `docker compose up -d --no-deps --force-recreate` for each service, one at a time.
3. Poll `/healthz` on the new container; abort and redeploy the previous tag if no 200 response within 60 seconds.
4. Old container is removed automatically by Docker once the new one passes health checks.

7.2 Rolling Update Flow Diagram



7.3 Rollback Procedure

A `rollback.sh` script accepts a previous image tag, re-runs `docker compose up` with that tag, and re-verifies health. The previous image is always retained on the host (not pruned automatically) so rollback requires no registry pull.

8. Observability

8.1 Stack

- **Prometheus** scrapes metrics from Node Exporter (host), cAdvisor (container resource usage), and application `/metrics` endpoints on every service.
- **Grafana** provides dashboards for infrastructure KPIs, container resource usage, HTTP latency, and error rates.
- **Alertmanager** routes alerts to a dedicated Slack channel via webhook; only actionable, critical conditions generate pages.

8.2 Key Alerts

Alert	Condition	Severity
HighCPU	CPU > 85% for 5 min	warning
HighMemory	Memory > 90% for 5 min	critical
ContainerDown	Container exits unexpectedly	critical
HighErrorRate	HTTP 5xx > 5% over 2 min	critical
DiskSpaceLow	Disk usage > 85%	warning
BackupMissing	No backup job success in 24 h	warning

8.3 Health Endpoints

Every service exposes a `GET /healthz` endpoint returning 200 OK with a JSON payload containing service name, version, and database connectivity status. Docker Compose `healthcheck` directives use this endpoint for native container health signalling.

9. Operational Procedures

9.1 Deploy

The deploy script pulls the target image tag, recreates each service via Docker Compose one at a time, polls `/healthz`, and logs the result. It uses `set -euo pipefail` throughout and exits non-zero on any failure.

9.2 Rollback

The rollback script accepts a previous tag as its only argument, re-runs the same Compose recreate sequence with that tag, and re-verifies health. No registry access is needed if the image is still cached locally.

9.3 Database Backup

A cron-scheduled script runs `pg_dumpall` inside the PostgreSQL container, compresses the output with `gzip`, and uploads it to a versioned S3 bucket using server-side KMS encryption. Local copies are deleted after successful upload.

9.4 Bootstrap

A `bootstrap.sh` script prepares a fresh Amazon Linux 2023 host: installs Docker and Docker Compose, configures the `docker` group, pulls the initial images from GHCR, and starts the stack. It is idempotent and safe to re-run.

10. Constraints & Trade-offs

Decision	Benefit	Limitation
Single EC2, no ASG	Low cost; simple ops	SPOF; vertical scale only
Docker Compose (not Kubernetes)	Minimal overhead	No native HA or HPA
Caddy (not Nginx)	Auto TLS, zero config	Less tunable than Nginx
GHCR (not ECR)	Free; co-located with code	Tied to GitHub account
Single-region deployment	Simplified IaC	No geo-redundancy
Rolling update (not blue-green)	No duplicate infra cost	Brief mixed-version window
OIDC (no static keys)	No credential leak risk	Trust policy maintenance
Trivy email reports	Passive audit trail every run	Requires SMTP relay setup

11. Future Improvements

1. **Kubernetes (EKS / k3s)** – replace Docker Compose for multi-replica, self-healing workloads.
2. **Blue-green or canary deployments** – eliminate the brief mixed-version window during rolling updates.
3. **Auto Scaling Group + ALB** – horizontal scaling and zero-downtime instance replacement.
4. **RDS PostgreSQL** – managed database with automated failover, PITR backups, and Multi-AZ.
5. **OpenTelemetry** – replace ad-hoc metrics with structured traces, metrics, and logs via OTLP to a centralised backend.
6. **Secrets Manager / Vault** – centralise secret rotation and maintain a full audit trail.
7. **Checkov / tfsec** – IaC security scanning integrated into the CI pipeline as an additional quality gate.
8. **SBOM generation** – attach a Software Bill of Materials (via `syft`) to every release for supply-chain auditing.
9. **GitOps (ArgoCD / Flux)** – declarative, pull-based deployments driven entirely from Git state.
10. **Staging environment** – a dedicated `dev` EC2 instance provisioned with the same Terraform workspace, receiving every push to `develop` automatically.